

How to Build a Simple Express Backend

A learning guide that overlays the core ideas onto a small **e-commerce backend** (products + orders). We explain *how* each piece works and *why* it exists. We use the `boston-warehouse-api` project (TypeScript + Express + Sequelize + MySQL) as the real-world reference for conventions, but keep the example code minimal so the concepts are easy to follow.

NOTE: You do NOT need to memorise every model/controller in that repo. The point is the pattern: how the folders are arranged, how a DB connection is made, how a model and `init-models` fit together, how controllers / utils / common are separated, what the request & response objects are, how middleware (including a global auth guard) works, linting, and how `run` vs `build` actually differ.

1. The big picture

A backend is a program that waits for HTTP requests, does work (talks to a database, runs business logic), and sends back an HTTP response. Three tools do the heavy lifting:

- **Express** — the web server. Maps a URL + HTTP verb (GET/POST/...) to a function, and hands you the request and response objects.
- **Sequelize** — an ORM. Reads/writes MySQL rows using JavaScript objects and methods instead of hand-written SQL.
- **TypeScript** — adds types so the compiler catches mistakes (wrong field names, mismatched shapes) before the code runs.

WHY? You could use raw Node `http` + manual SQL, but then you re-build routing, connection pooling and row-to-object mapping yourself. Express + Sequelize give those for free so you focus on business logic.

2. Folder structure — and why it is arranged this way

A predictable layout is what lets a team grow without chaos. A clean e-commerce backend:

```
ecommerce-api/  
|- .env                # secrets (NEVER commit)  
|- .gitignore          # what git must NOT track  
|- package.json        # dependencies + scripts  
|- tsconfig.json       # TypeScript rules  
|- src/  
  |- index.ts          # ENTRY POINT: creates the Express app  
  |- config/           # db connection wiring  
  |- middleware/       # auth, upload (run on the way IN)  
  |- utils/            # shared helpers (responses, user from token)  
  |- modules/  
    |- shop/  
      |- models/       # Sequelize models + init-models.ts  
      |- controllers/  # HTTP handlers (thin)  
      |- common/       # reusable business logic  
      |- route/        # URL -> controller mapping
```

Why group by feature, not by type?

- One folder per feature keeps everything about 'shop' in one place — the blast radius of a change is obvious.
- **models/** = tables. **controllers/** = HTTP handlers. **common/** = the logic two or more controllers reuse. **route/** = thin glue from URL to controller.
- **utils/** = helpers many features share (response envelope, decode token). **middleware/** = functions that run before the handler (auth, upload).

WHY? The single most important idea: **controllers stay thin**. A controller parses the request, calls a `common/` function, returns a response. The real logic lives in `common/` so it can be reused and tested without an HTTP request.

3. package.json — the control panel

Declares dependencies and the commands (scripts) to run things.

```
{
  "type": "commonjs",
  "scripts": {
    "start": "nodemon --exec ts-node src/index.ts", // dev, reload on save
    "start:prod": "node dist/index.js",             // prod, compiled JS
    "build": "npm run lint && tsc",                  // check then compile
    "lint": "eslint src --fix"
  },
  "dependencies": { "express", "sequelize", "mysql2", "cors", "jsonwebtoken" },
  "devDependencies": { "typescript", "ts-node", "eslint" } // build-only
}
```

Why dependencies vs devDependencies?

At deploy, `npm ci --only=production` installs only dependencies. TypeScript and ESLint are not needed at runtime, so they stay in `devDependencies` and keep the production image small.

WHY? `type: commonjs` must match `tsconfig`'s `module` setting. If they disagree, the build breaks. Keep them consistent.

4. .gitignore — keep junk and secrets out of git

- **Secrets:** `.env` holds DB passwords / API keys. Committing it leaks credentials to everyone with repo access.
- **Generated output:** `node_modules/` and `dist/` are recreated by `npm install` / `npm run build`. They are huge and change constantly — no value in git.
- **Noise:** `*.log`, `.DS_Store`, `.eslintcache`.

```
# .gitignore (the lines that matter)
node_modules/
dist/
.env
.env.test
*.log
.DS_Store
```

NOTE: Secrets are injected at deploy time, not stored in the repo. Locally you create your own `.env` with your DB credentials.

5. Connecting to the database

`config/` reads settings from env vars (same code runs against dev/prod) and creates one Sequelize instance. A connection **pool** reuses open connections instead of opening a new one per request.

```
// config/db.config.ts
export const dbConfig = {
  HOST: process.env.DB_HOST, USER: process.env.DB_USER,
  PASSWORD: process.env.DB_PASS, DB: process.env.DB_SCHEMA,
  dialect: "mysql",
  pool: { max: 10, min: 0, acquire: 30000, idle: 10000 },
};

// config/index.ts
```

```
export const sequelizeDB = new Sequelize(
  dbConfig.DB, dbConfig.USER, dbConfig.PASSWORD,
  { host: dbConfig.HOST, dialect: "mysql", pool: dbConfig.pool });
```

WHY? Pool sizing: max = most simultaneous connections, acquire = max ms to wait for a free one, idle = how long an unused one lives before closing.

6. Models & init-models

Each table is a model: it declares the TypeScript *shape* of a row and the column types. `init-models.ts` is the registry that binds every model to the connection.

A model (e-commerce: Product)

```
export interface productAttributes {
  id: number; name: string; price: number; stock?: number;
}
export type productCreationAttributes =
  Optional<productAttributes, "id" | "stock">; // auto-filled

export class product extends Model<productAttributes, productCreationAttributes>
  implements productAttributes {
  declare id: number; declare name: string; declare price: number;
  static initModel(sequelize: Sequelize) {
    return product.init(
      { id: { primaryKey: true, autoIncrement: true, type: DataTypes.INTEGER },
        name: { type: DataTypes.STRING, allowNull: false },
        price: { type: DataTypes.DECIMAL(10,2) }, stock: { type: DataTypes.INTEGER } },
      { sequelize, tableName: "product", timestamps: true });
  }
}
```

The 5 touch-points of `init-models.ts`

When you add a model, touch five places so the whole app can see it:

- 1. value import: `import { product as _product } from "../product";`
- 2. type import: `import type { productAttributes, productCreationAttributes } from "../product";`
- 3. re-export value: `_product as product`, in the `export {}` block
- 4. re-export types: add both `*Attributes` types to `export type {}`
- 5. register + init: `const product = _product.initModel(sequelize);` then include it in the returned object

WHY? Why a central registry? Controllers import from `init-models`, not the raw file. That guarantees the model is already bound to the connection — no 'model not initialized' errors — and gives one place to see all tables.

NOTE: Gotcha the team hit: optional (?) fields are typed `string | undefined`, NOT `string | null`. Passing `null` causes a TS error — use `value || undefined`.

7. Controllers, utils and common

Controllers — thin HTTP handlers

An async function (`req`, `res`) that validates input, calls business logic, and shapes the JSON. Example: get one product.

```
export async function getProduct(req: Request, res: Response) {
```

```

try {
  const { id } = req.params; // URL /:id
  const p = await product.findById(Number(id));
  if (!p) return res.json({ status: false, code: "NOT_FOUND" });
  return res.json({ status: true, code: "SUCCESS", data: p });
} catch (e) { return res.json(errorMessageHandle(e)); }
}

```

utils — cross-cutting helpers

utils/ holds functions many features share, e.g. a standard response envelope ({status, code, message, data}) and a getUserId(req) that decodes the JWT. Uniform responses make the API predictable to consume.

common — reusable business logic

common/ holds the real work as plain async functions with no HTTP dependency. Example: placeOrder() — check stock, create an order, decrement stock. Both a REST controller and a background job can call it.

WHY? Don't duplicate logic. If a flow already exists in a controller, extract the core into common/ and call it from both places. The handler stays a thin orchestrator.

8. Request & response objects

- **req** — the incoming call: req.params (URL segments), req.query (?x=1), req.body (JSON payload), req.headers (JWT), req.file (upload).
- **res** — how you reply: res.status(200).json({...}) or res.send(...); res.write() streams chunks.
- **next** — hand control onward; next(err) jumps to the error handler.

```

app.use(express.json()); // parse JSON body -> req.body
app.use(cors()); // allow browser calls from other origins
app.use("/v1", shop); // mount routes under /v1

```

NOTE: Many APIs (including this repo) return HTTP 200 even on error and put the real result in a JSON status_code field. Clients read that field, not the HTTP number.

9. Middleware — and adding a global auth guard

Middleware is (req, res, next) that runs *before* your handler. The repo uses it for file uploads and a per-request context store. Auth, though, is done per-route today (each controller calls getUserId) — which means a forgotten check leaves a route open. The better design is one global guard.

Add a global auth middleware

```

// middleware/auth.ts
export function authMiddleware(req, res, next) {
  const header = req.headers.authorization;
  if (!header) return res.status(401).json({ code: "NO_TOKEN" });
  try {
    req.user = jwt.verify(header, process.env.JWT_SECRET); // attach user
    next(); // allowed -> continue
  } catch { return res.status(401).json({ code: "BAD_TOKEN" }); }
}

// index.ts -- one line protects EVERY /v1 route
app.use("/v1", authMiddleware, shop);

```

WHY? Why global? One line protects all routes. Controllers no longer have to remember the check; the user is already on req.user. Less duplication, fewer open endpoints.

The one public exception

A public endpoint (e.g. an upload that should work anonymously) is mounted on a separate router without the guard, and tolerates a missing token by skipping the user-specific step:

```
const publicRouter = Router();
publicRouter.post("/upload", upload.single("file"), handleUpload);
app.use("/v1", authMiddleware, shop); // protected
app.use("/v1", publicRouter); // open
```

10. Linting — consistent style, automatically

ESLint reads a config and flags style/bug problems. `eslint src --fix` auto-fixes what it can. Because `build = lint && tsc`, a lint failure fails the build, so bad style can't reach production.

```
// eslint.config.mjs (essentials)
rules: {
  "dot-notation": "error", // obj.x not obj['x']
  "no-unused-vars": "error", // dead code
  "@typescript-eslint/no-explicit-any": "off" // repo allows 'any'
}
```

WHY? Linting is a team contract. It removes style debates and catches real bugs (unused vars, unreachable code, `==` vs `===`). Run `npm run lint` any time.

11. Run vs build

Development — hot reload

`npm run start` uses nodemon + ts-node: runs TypeScript directly (no compile) and restarts on save. Best while writing code.

Build — check then compile

`npm run build` runs `lint && tsc`: ESLint first, then the compiler emits JavaScript into `dist/` (per `tsconfig`). You serve that with `npm run start:prod` (node `dist/index.js`).

```
// tsconfig.json
{ "compilerOptions": { "target": "es2022", "module": "node16",
  "outDir": "dist", "rootDir": "src", "strict": true },
  "include": ["src/**/*"] }
```

Production — Docker

A two-stage Dockerfile installs + compiles in stage 1, then copies only `dist/` + production packages into stage 2 and runs as a non-root user — small, safe image.

```
FROM node:24-alpine AS builder
COPY . . RUN npm ci RUN npm run build # -> dist/
FROM node:24-alpine
COPY --from=builder /app/dist ./dist
RUN npm install --only=production
USER node
CMD ["node", "dist/index.js"]
```

WHY? Mental model: **write** in TS with nodemon → **check** with lint+tsc (build) → **ship** compiled `dist/` in Docker. The `.ts` files are source; `dist/` is what actually runs.

12. Assignment

- Scaffold an e-commerce API: `package.json` with `deps/devDeps` + `start/build/lint` scripts; a `.gitignore` that excludes `.env`, `node_modules`, `dist` (explain why).

- Add `config/` building a pooled Sequelize instance from env vars.
- Define a `Product` and an `Order` model; register both in `init-models` (all 5 touch-points).
- Split into `controllers/` (thin), `common/` (`placeOrder`), `utils/` (response envelope).
- Add a global `authMiddleware` mounted on `app.use('/v1', auth, router)`, with one public route.
- Run `npm run build`; confirm `dist/` is produced and explain dev vs prod run.

NOTE: Reuse a standard response envelope so every endpoint returns `{status, code, message, data}` consistently — that uniform shape is what makes an API pleasant to consume.

Learning aid. Concepts and conventions reference the live `boston-warehouse-api` (Express/TS/Sequelize); the e-commerce snippets are illustrative.